```cpp
#pragma once
#include <vector>
#include <cmath>
#include "matrix.hpp"

class PositionalEncoding {
private:
    size_t maxSeqLen_;
    size_t dimModel_;
    Matriz peMatrix_; // Guardará la matriz fija de posiciones (dimModel x maxSeqLen)

public:
    PositionalEncoding(size_t maxSeqLen, size_t dimModel)
        : maxSeqLen_(maxSeqLen), dimModel_(dimModel), peMatrix_(dimModel, maxSeqLen, 0.0)
    {
        // Generar la matriz sinusoidal del paper "Attention Is All You Need"
        for (size_t pos = 0; pos < maxSeqLen_; ++pos) {
            for (size_t i = 0; i < dimModel_; i += 2) {
                double divTerm = std::pow(10000.0, static_cast<double>(i) / static_cast<
double>(dimModel_));

                // Par (2i) -> Seno
                peMatrix_(i, pos) = std::sin(static_cast<double>(pos) / divTerm);

                // Impar (2i+1) -> Coseno
                if (i + 1 < dimModel_) {
                    peMatrix_(i + 1, pos) = std::cos(static_cast<double>(pos) / divTerm);
                }
            }
        }
    }

    // Retorna el vector de posición para un índice temporal 'pos' como Matriz (dimModel
x 1)
    Matriz obtenerPosicion(size_t pos) const {
        if (pos >= maxSeqLen_) {
            throw std::out_of_range("Posición fuera de la longitud máxima de secuencia."
);
        }

        Matriz vecPos(dimModel_, 1);
        for (size_t d = 0; d < dimModel_; ++d) {
            vecPos(d, 0) = peMatrix_(d, pos);
        }
        return vecPos;
    }

    // Imprime la tabla precalculada
    void imprimirTabla() const {
        peMatrix_.imprimir("Tabla_PositionalEncoding");
    }
};
```